

Fourier Coefficients Evaluation using Python

```
In [1]: import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import quad
```

```
In [2]: # function
def f(x):
    return x

# half period
L = np.pi
# no of fourier terms
N = 50
```

```
In [3]: # fourier coeff
def a0():
    val, _ = quad(lambda x: f(x), -L, L)
    return val/L

def an(n):
    val, _ = quad(lambda x: f(x)*np.cos(n*np.pi*x/L), -L, L)
    return val/L

def bn(n):
    val, _ = quad(lambda x: f(x)*np.sin(n*np.pi*x/L), -L, L)
    return val/L
```

```
In [4]: # value of coeff
a0_val = a0()
a_val = [an(n) for n in range(1, N + 1)]
b_val = [bn(n) for n in range(1, N + 1)]

print(a0_val, a_val, b_val)
```

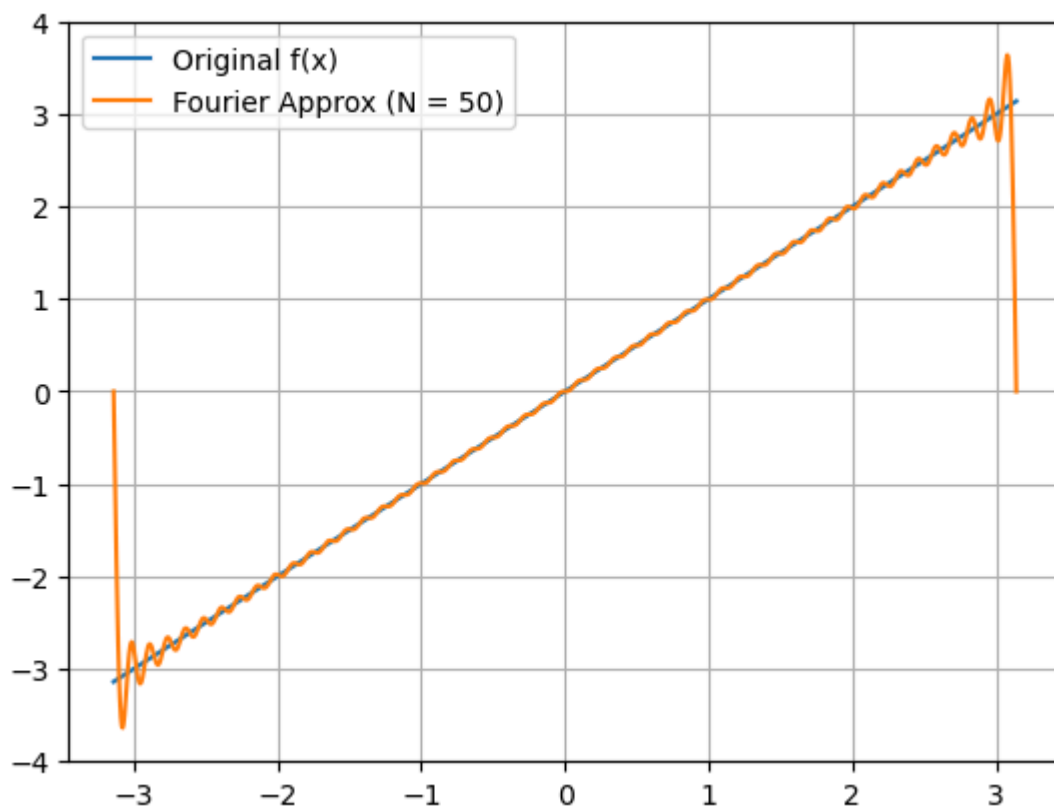
```
0.0 [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
0.0, 0.0, 0.0, 0.0, 0.0] [2.0000000000000004, -0.9999999999999997, 0.
6666666666666669, -0.4999999999999997, 0.39999999999999974, -0.333333333333
33334, 0.28571428571428453, -0.2499999999999995, 0.22222222222222182, -0.1
99999999999999965, 0.18181818181818185, -0.16666666666666627, 0.15384615384
615388, -0.1428571428571435, 0.13333333333333333, -0.12499999999999956, 0.1
1764705882352922, -0.111111111111111256, 0.10526315789473728, -0.1000000000
0000156, 0.09523809523809713, -0.09090909090909195, 0.08695652173913203, -
0.083333333333333444, 0.079999999999999884, -0.07692307692307533, 0.07407407
407407275, -0.07142857142857004, 0.06896551724137785, -0.06666666666666683
2, 0.06451612903225944, -0.06249999999999968, 0.06060606060606063585, -0.058
8235294117677, 0.05714285714285695, -0.05555555555555533, 0.05405405405405
405, -0.052631578947368834, 0.05128205128205242, -0.0500000000000000565, 0.
04878048780487998, -0.04761904761904945, 0.04651162790697673, -0.045454545
45454822, 0.0444444444444444814, -0.043478260869567145, 0.04255319148936293
6, -0.041666666666666789, 0.04081632653061259, -0.039999999999999776]
```

```
In [5]: # Reconstruct functions
x_vals = np.linspace(-L, L, 1000)
f_fourier = np.full_like(x_vals, a0_val/2)

for n in range(1, N+1):
    f_fourier += a_val[n-1]*np.cos(n*np.pi*x_vals/L) + b_val[n-1]*np.sin(

f_original = np.array([f(x) for x in x_vals])
```

```
In [6]: plt.plot(x_vals, f_original, label="Original f(x)")
plt.plot(x_vals, f_fourier, label = f"Fourier Approx (N = {N})")
plt.legend()
plt.grid(True)
plt.show()
```



```
In [ ]:
```